

Rota Vital — Atividade em Equipe: Threads II

Cruzamento **requisições × estoque** em escala nacional: versão sequencial vs. versões com threads

Equipe: João Lucas Santos Lira, Mariana Ferreira, Thaissa Fernandes, Priscila Maciel de Lima |
Professor: Raoni Monteiro de Oliveira | **Data:** 22/09/2026

Código: <https://github.com/joaosanli/rota-vital-threads> (Java 21 + Spring Boot 3)

1. Justificativa

1.1 Como encontramos a operação

Seguimos o roteiro: listamos as operações da camada de aplicação que processam volume e, para cada uma, perguntamos onde o tempo é gasto na escala nacional, qual a Big-O e se os dados se partem em fatias independentes.

Operação candidata	Onde vai o tempo (escala nacional)	Big-O	Particionável?	Serve?
Estatísticas do histórico (totais e médias por UF/mês)	Ler milhões de linhas; a conta é trivial. GROUP BY + índices ou visão materializada no banco resolvem.	$O(n)$	Sim	Não — melhorar o SQL
Deteção de duplicatas	Com hash ou índice único a checagem é $O(1)$ por registro; o custo é E/S.	$O(n)$	Parcialmente	Não — UNIQUE/índice
Ordenação da fila por prioridade	ORDER BY sobre índice, ou heap mantido incrementalmente a cada nova requisição.	$O(n \log n)$	Sim	Não — índice/heap
Validações em lote	Regras simples por registro; domina ler e gravar o lote.	$O(n)$	Sim	Ganho pequeno
Cruzamento requisições × estoque (sugerir o melhor fornecedor para cada requisição pendente)	Cálculo: cada requisição é comparada com todos os hospitais que têm o insumo — distância geodésica (senos, raiz, arco-seno) + regras de estoque por par.	$O(R \cdot H)$	Sim — requisições independentes	Sim — escolhida

1.2 A operação escolhida

O painel nacional do Rota Vital precisa responder, várias vezes ao dia: *para cada requisição de insumo pendente, qual hospital deveria fornecer?* Para uma requisição (local, insumo, quantidade q , prioridade), o serviço percorre os hospitais que têm o insumo, descarta os com estoque $< q$, calcula a distância (Haversine) e uma pontuação = distância + penalidade de 80 km se o fornecedor ficaria com menos que $2q$, e escolhe a menor (empate: menor id). Em seguida agrega indicadores: requisições atendíveis, distância média, % dentro do SLA por prioridade (1 h, 4 h, 12 h, 48 h), entregas interestaduais e os mesmos números por UF.

1.3 Big-O da solução sequencial

Com R requisições pendentes e C hospitais candidatos por insumo (C proporcional a H , o total de hospitais), o laço externo roda R vezes e o interno C vezes: **$O(R \cdot C) = O(R \cdot H)$** em tempo, $O(R + H \cdot I)$ em memória ($I = n^\circ$ de insumos). Na nossa base, $H = 1.000$ hospitais, $I = 30$ insumos e C médio = 209,6; com $R = 1.000.000$ são ~210 milhões de pares avaliados, cada um com funções trigonométricas. Dobrar R ou H dobra o tempo.

1.4 Onde está o gargalo

Os dados de entrada são lidos uma vez ($O(R)$ de E/S, uma consulta simples) e ficam em memória; o tempo depois disso é todo de CPU dentro do laço $O(R \cdot H)$. Gerar/carregar o snapshot de 1.000.000 requisições levou 127,4 ms, enquanto o cruzamento sequencial levou 12.121,1 ms. Por que não é caso de “melhorar o SQL”: um CROSS JOIN com a fórmula de distância faria os mesmos $R \cdot C$ cálculos dentro do banco — o recurso mais caro de escalar — e travaria as demais consultas do sistema; índices comuns não ajudam, porque a pontuação depende do par (requisição, hospital) e do estoque. (Um índice espacial poderia podar candidatos — otimização algorítmica complementar ao paralelismo, não substituta.)

1.5 Por que os dados são particionáveis

O cálculo de uma requisição só lê dados imutáveis durante a operação (hospitais e snapshot do estoque) e só produz o resultado dela mesma: nenhuma requisição depende de outra. A sugestão não reserva estoque — a reserva acontece depois, numa transação, quando o regulador confirma. Portanto $[0, R)$ pode ser cortado em p fatias contíguas, cada thread processa a sua com o próprio acumulador, e os resultados parciais são somados no fim. Como a agregação é soma de inteiros (distâncias em metros inteiros) — associativa e

comutativa —, a ordem em que as fatias terminam não altera nada: a resposta paralela é **idêntica** à sequencial.

2. O serviço (Java 21 + Spring Boot)

A operação é um endpoint REST real: a requisição HTTP chega ao *CruzamentoController* (thread do Tomcat), que chama o *CruzamentoService*; este obtém o snapshot do *RepositorioDeDados* e executa o *MotorCruzamento* no modo pedido. Nas versões com threads o motor fatia $[0, R)$, submete uma tarefa por fatia a um *ExecutorService* (pools fixos de 2, 4 e 8 threads de plataforma, criados uma vez e reaproveitados; ou um executor de virtual threads), espera todas com *invokeAll* (join), soma as parciais e devolve o JSON.

```
POST /api/dados/gerar?requisicoes=1000000 # gera a base (semente fixa = 42)
GET /api/cruzamento?modo=sequencial&requisicoes=1000000
GET /api/cruzamento?modo=plataforma&threads=8&requisicoes=1000000
GET /api/cruzamento?modo=virtual&threads=8&requisicoes=1000000
GET /api/cruzamento?modo=inseguro&threads=8&requisicoes=1000000 # demo de race condition
```

Mesma resposta em todas as versões. Cada fatia usa um acumulador próprio (confinamento de thread: nada compartilhado é escrito durante o cálculo, então não há lock nem disputa); só a thread da requisição faz a agregação. A resposta traz um *checksum* de todos os pares (requisição, hospital escolhido) e uma *assinatura* de todos os números; o benchmark compara a assinatura de cada chamada com a da sequencial e **todas foram iguais**. Um teste JUnit verifica o mesmo para 2, 3, 4, 8 e 16 threads. Para contraste, o modo *inseguro* faz N threads somarem num único acumulador compartilhado sem sincronização ($x++$ = ler, somar, gravar): incrementos se perdem. Na nossa execução, 3 de 3 tentativas do modo inseguro divergiram da sequencial — e o resultado muda de uma execução para outra, típico de race condition.

3. Medições

Ambiente: Spring Boot + Tomcat embutido; 12 núcleo(s) disponível(is) para a JVM; Java 21.0.12; Windows 11 amd64; heap máx. 2048 MB. **Método:** tempo de resposta HTTP medido no cliente (benchmark.py), 1 chamada de aquecimento do JIT por configuração descartada, 3 medições intercaladas por configuração, valor = **mediana**. $\text{Speedup} = T_{\text{seq}} / T_p$; eficiência = speedup / n° de threads.

Requisições	Versão	Threads	Resposta HTTP (ms)	Processamento (ms)	Speedup	Eficiência
100.000	sequencial	1	483,4	466,3	1,00×	100%
100.000	plataforma	2	262,2	247,0	1,84×	92%
100.000	plataforma	4	196,6	171,5	2,46×	61%
100.000	plataforma	8	123,6	108,0	3,91×	49%
100.000	virtual	2	257,1	242,3	1,88×	94%
100.000	virtual	4	187,3	167,8	2,58×	65%
100.000	virtual	8	123,1	101,5	3,93×	49%
1.000.000	sequencial	1	12.143,3	12.121,1	1,00×	100%
1.000.000	plataforma	2	7.420,9	7.393,8	1,64×	82%
1.000.000	plataforma	4	3.222,4	3.205,8	3,77×	94%
1.000.000	plataforma	8	1.806,5	1.802,2	6,72×	84%
1.000.000	virtual	2	3.428,8	3.401,9	3,54×	177%
1.000.000	virtual	4	2.504,1	2.499,9	4,85×	121%
1.000.000	virtual	8	1.840,9	1.811,5	6,60×	82%

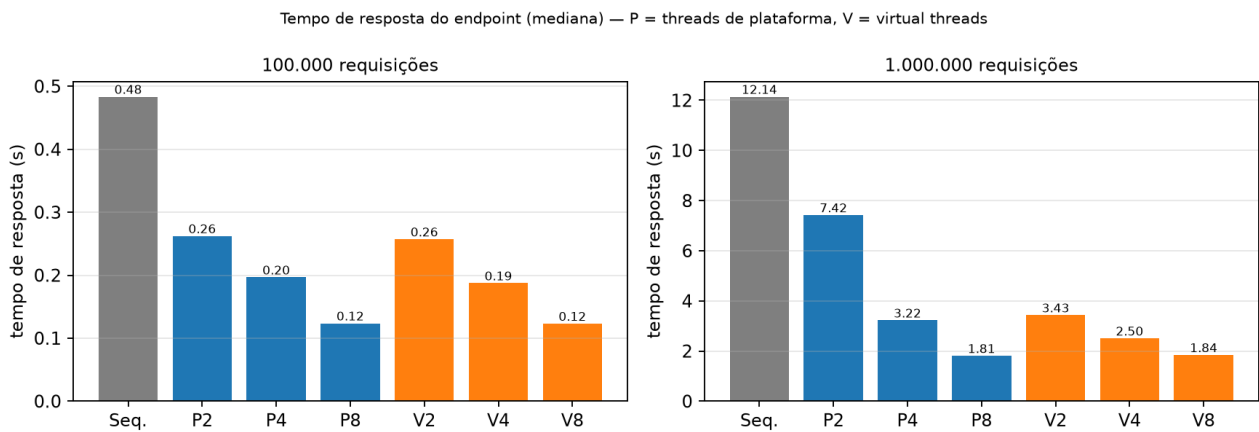


Figura 1 — Tempo de resposta do endpoint por versão e tamanho de entrada.

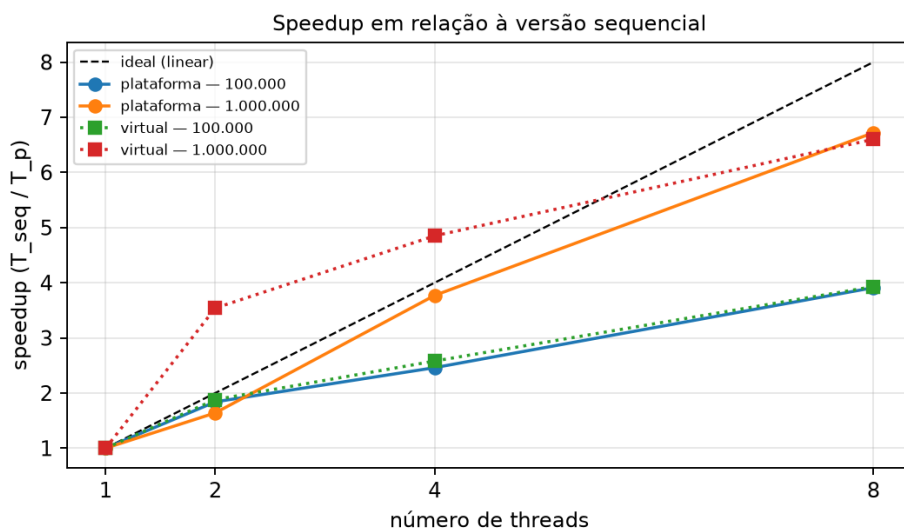


Figura 2 — Speedup × número de threads (linha tracejada = speedup ideal).

4. Análise

O ganho foi linear? Não: speedup de 1,64× (2), 3,77× (4), 6,72× (8) com 1.000.000 requisições, em 12 núcleos lógicos. Pela Lei de Amdahl, receber o HTTP, fatiar, agregar e serializar o JSON seguem sequenciais; há custo de coordenação (submeter e esperar todas); as fatias têm custo desigual (insumos raros têm menos candidatos) e a resposta espera a mais lenta; núcleos lógicos (hyper-threading) dividem unidades de cálculo e o turbo cai com todos ocupados. Com 100.000 requisições foi 3,91× em 8 threads: o custo fixo pesa mais.

A Big-O mudou? Não. O trabalho segue $O(R \cdot H)$ — os mesmos pares são avaliados, só repartidos. O tempo ideal vira $O(R \cdot H/p) + O(p)$ da agregação, mas p é constante: threads dividem o tempo por uma constante, não mudam a classe. Tanto que 10× mais requisições custaram 25,1× mais tempo.

Concorrência × paralelismo. Na Mesa DJ as threads passavam o tempo dormindo e reagindo a eventos: *concorrência*, lidar com várias coisas ao mesmo tempo — rodaria igual num único núcleo. Aqui elas dividem um volume de cálculo sem esperar nada: *paralelismo*, fazer várias coisas ao mesmo tempo, que só acelera com núcleos livres para executá-las simultaneamente.

Quando nem 8 threads bastarem. Uma máquina para no número de núcleos dela; o passo seguinte é escalar *horizontalmente*: tornar o cruzamento um job assíncrono (a API responde 202 e o painel busca o resultado); distribuir as fatias (por UF ou faixa) entre *workers* em várias máquinas via fila de mensagens, com um agregador somando as parciais — o mesmo fatiar-e-agregar, agora distribuído (map/reduce); replicar a API stateless atrás de um balanceador; e recalculer só o que mudou. Esse salto de threads para processos em várias máquinas é o gancho da Unidade 2.

5. Opcional — virtual threads (Java 21) × threads de plataforma

Com 8 virtual threads e 1.000.000 requisições o endpoint levou 1.840,9 ms, contra 1.806,5 ms com 8 threads de plataforma (+1,9%) — na prática, empate. É o esperado: virtual threads são executadas sobre um pequeno pool de *carrier threads* (ForkJoinPool com paralelismo = nº de núcleos, aqui 12). Numa tarefa CPU-bound elas nunca bloqueiam, logo nunca liberam o carrier, e o paralelismo real continua limitado pelos núcleos. Virtual threads brilham em cargas de E/S (milhares de requisições esperando banco ou rede — o perfil da Mesa DJ e da própria API atendendo usuários), onde tornam barato ter uma thread por requisição; para acelerar cálculo, o que importa é o número de núcleos, não o tipo de thread.

6. Como reproduzir

```
./mvnw spring-boot:run      # ou rodar RotaVitalApplication pela IDE (Java 21)
python3 scripts/benchmark.py --tamanhos 100000,1000000 --reps 3
pip install matplotlib reportlab
python3 scripts/gerar_relatorio.py --entrada resultados
```

Os dados são gerados com semente fixa: qualquer máquina produz exatamente as mesmas requisições, hospitais e resposta (mesma assinatura); só os tempos mudam.